

Appunti del corso di Linguaggi

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

Indice

1	Introduzione	1
1.1	Perchè studiare i linguaggi	1
1.2	Cosa sono i linguaggi di programmazione	2
1.3	Cosa sono i linguaggi di programmazione	3
1.4	Contesti di sviluppo	3
1.5	Classificazione dei linguaggi	3
1.5.1	Linguaggi a basso livello	4
1.5.2	Linguaggi ad alto livello	4
1.6	Implementazione di un linguaggio	4
2	Descrivere i linguaggi	5
2.1	Sintassi	5
2.1.1	Grammatiche context-free	7
2.1.2	Notazione BNF	7
2.1.3	Definizione non formale	8
2.2	Analisi semantica	9
2.2.1	Induzione strutturale	9
2.2.2	Semantica	10
3	Espressioni	11
3.1	Caratteristiche delle espressioni	11
3.1.1	Notazione	11
4	Dichiarazioni	12
4.1	Identificatori	12
4.1.1	Aspetti progettuali	12
4.1.2	Bindings - legami	13
4.1.2.1	Bindings nei PL	14
4.1.2.2	Tipi di bindings	14
4.1.2.3	Tempi di bindings	14
4.2	Semantica delle dichiarazioni	14

Introduzione

Il problema dell'informatica consiste nel voler far eseguire a una macchina algoritmi per la manipolazione dei dati.

Il primo meccanismo che viene associato alla storia dei computer è la **macchina di Babbage**, che può essere visto come il primo "computer" perchè nasce come macchina per eseguire calcoli matematici complessi in modo automatico.

Il salto si verifica quando si decide che la macchina debba essere programmabile, cioè deve eseguire ciò che gli viene detto.

In questo senso nasce **Enigma** che nasce per decifrare un codice mediante un algoritmo di forza bruta e prova tutte le combinazioni possibili.

La prima architettura che darà origine alla struttura moderna dei computer è l'**architettura di Von Neumann**.

Si tratta di una tipologia di architettura hardware che è in grado di interpretare solo **linguaggio binari** (stringhe di bit).

Questo fa sì che l'uomo per poter interagire con la macchina ci sia un'evoluzione del linguaggio, ovvero bisogna costruire un linguaggio che sia più comprensibile all'essere umano.

1.1 Perchè studiare i linguaggi

Lo studio dei linguaggi di programmazione (PL) permette di:

- ▷ **migliorare la capacità di esprimere idee in forma algoritmica**
I programmatori, infatti, sono vincolati ai linguaggi che già conoscono e non sfruttano il fatto che i linguaggi di programmazione si sono evoluti per rispondere a certe esigenze in modo efficace ed efficiente
- ▷ **scegliere il linguaggio di programmazione appropriatamente**
- ▷ **studiare/progettare più agevolmente nuovi linguaggi**
Una volta acquisiti questi concetti diventa più semplice capire come questi concetti sono incorporati nel linguaggio che si sta studiando
- ▷ **comprendere meglio il significato ed efficienza dei linguaggi di programmazione**

1.2 Cosa sono i linguaggi di programmazione

L'evoluzione dei linguaggi di programmazione dipende strettamente dal tipo di architettura di Von Neumann adottata.

L'**architettura di Von Neumann** è una **macchina programmabile con programma memorizzato**, dove dati e istruzioni condividono la stessa area di memoria (prima era esterno [es. schede perforate])

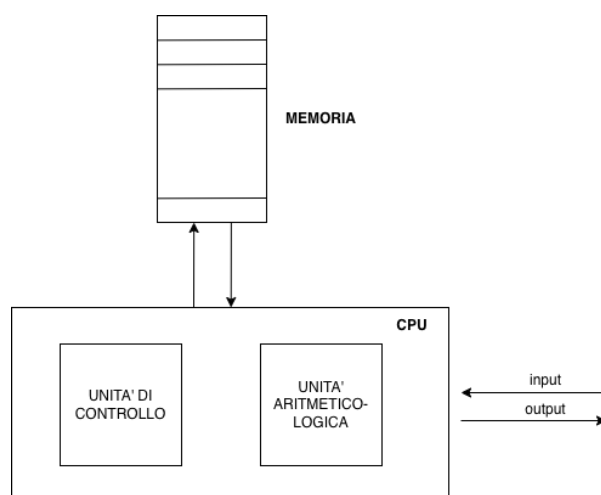


Figura 1.1: architettura di Von Neumann

Il linguaggio di programmazione, quindi, per poter operare su questa architettura, deve istruire una CPU per eseguire i calcoli (algoritmi) su dati conservati in memoria.



Figura 1.2: linguaggio di programmazione

Da questa definizione, ne deriva che, anche il linguaggio binario è un linguaggio di programmazione, ma non è umanamente comprensibile perché dati e istruzioni sono rappresentati nello stesso modo.

Un linguaggio di programmazione deve permettere di superare questa difficoltà.

Definizione: algoritmo

Un **algoritmo** è una sequenza passi, il cui scopo è il calcolo di una funzione.

Un algoritmo è un concetto astratto che trova forma concreta in un **programma**.

Definizione: programma

Un **programma** è una sequenza (finita) di istruzioni.

I **linguaggi di programmazione** nascono, quindi, proprio per questo motivo: linguaggi formali le cui frasi sono programmi, ovvero forme concrete di algoritmi eseguibili/interpretabili da una macchina.

1.3 Cosa sono i linguaggi di programmazione

Il programma è la concretizzazione di un algoritmo che manipola i dati. manca, però, un altro ingrediente: la **trasformazione** dei dati che l'algoritmo vuole eseguire. Questa trasformazione è il significato dell'algoritmo, la semantica del programma.

- ▷ **programma**: costituisce la sintassi
- ▷ **effetto dell'esecuzione del programma** (trasformazione dei dati): costituisce la semantica

Il programma è necessariamente un oggetto finito (sempre concreto), mentre il calcolo di una funzione richiede l'esecuzione di un insieme di passi primitivi, che però possono essere infiniti. Questo significa che bisogna pensare al programma come a una rappresentazione finita di un insieme potenzialmente infinito, di passi primitivi di computazione.

I linguaggi di programmazione devono dare specifica finita di oggetti potenzialmente infiniti.

1.4 Contesti di sviluppo

Il contesto in cui il linguaggio deve operare determina le funzionalità che il linguaggio deve gestire efficientemente:

- ◇ **applicazioni scientifiche**: computazione floating-point, utilizzo array (es. **Fortran**)
- ◇ **applicazioni economiche**: produrre report, usare numeri decimali e caratteri (es. **COBOL**)
- ◇ **intelligenza artificiale**: manipolazione di simboli (es. **LISP**)
- ◇ **programmazione di sistemi**: necessita efficienza per l'uso continuo (es. **C**)
- ◇ **web software**: linguaggi markup (es. **HTML**), scripting (es. **PHP**), general-purpose (es. **Java**)

Nuove metodologie e nuove esigenze hanno portato all'estensione di linguaggi esistenti, ma anche allo sviluppo di nuovi paradigmi e nuovi linguaggi di programmazione.

1.5 Classificazione dei linguaggi

1.5.1 Linguaggi a basso livello

I linguaggi a basso livello sono più vicini al linguaggio macchina e hanno caratteristiche specifiche dipendenti dall'architettura che si sta programmando:

- ▷ **linguaggio binario**
- ▷ **Assembly**

1.5.2 Linguaggi ad alto livello

I linguaggi di alto livello si possono classificare in base:

- ▷ **classificazione per paradigma di programmazione:**
 - ◇ **procedurale**: codice diviso in sub-routine che eseguono compiti semplici basato su molti condizionali [GOTO] (es. COBOL)
 - ◇ **strutturali (imperativo)**: procedurale senza goto (es. C, Pascal, Ada)
 - ◇ **OOP**: si basa sul concetto di classe (es. Java, C++, c#).
Un linguaggio OOP deve avere tre requisiti:
 - * incapsulamento
 - * eridetarietà
 - * polimorfismo
 - ◇ **dichiarativo**: basati fortemente sulla matematica
 - * logici: si basano sulla logica proposizionale del primo ordine (dimostrazioni di fatti) (es. PROLOG)
 - * funzionali: si basano su applicazioni di funzioni (es. LISP, ML)
- ▷ **classificazione generazionale**
 - I* linguaggi di basso livello, linguaggi proprietari
 - II* linguaggi di basso livello con prima astrazione (es. Assembly)
 - III* linguaggi di alto livello con primo approccio verso il linguaggio naturale (es. C)
 - IV* linguaggi dichiarativi per poter manipolare i dati in modo matematico (es. SQL)
 - V* non semplice esecuzione di programmi scritti e fissati (AI e linguaggi dinamici)

1.6 Implementazione di un linguaggio

Descrivere i linguaggi

Un linguaggio di programmazione non è altro che come un linguaggio naturale, ma semplificato e non ambiguo.

La descrizione di un linguaggio avviene su tre dimensioni:

- ▷ **sintassi**: insieme delle regole che permettono di costruire frasi corrette.
 Descrive la relazione tra simboli.
 La fase che determina la corretta struttura delle frasi, ovvero che verifica se una frase rispetta le regole della grammatica è chiamata **parsing**.
- ▷ **semantica**: attribuzione di un significato ai simboli (cosa vuol dire una frase corretta).
 Descrive la relazione tra simboli e significato.
- ▷ **pragmatica**: insieme delle regole di buon utilizzo di un linguaggio di programmazione (qual'è il modo corretto di usare frasi corrette e sensate).
 Descrive la relazione tra segni, simboli e utente.

Caratteristiche generali di qualsiasi PL

- ▷ **implementazione**: strumenti di esecuzione di frasi ben formate dal linguaggio di programmazione

2.1 Sintassi

Le regole sintattiche del linguaggio specificano quali stringhe di caratteri sono legali nel linguaggio:

- ◇ **parola**: stringa di caratteri su un alfabeto Σ
- ◇ **frase**: sequenza di parole (ben formate)
- ◇ **linguaggio**: insieme di frasi (ben formate)

Esempio: linguaggi VS fondamenti

$$\{a^n b^n \mid n \in \mathbb{N}\}$$

⇒ **alfabeto**:

$$\Sigma = \{a, b\} \text{ — Fondamenti}$$

$$\mathcal{F} = \{\text{if, while, then, else, ...}\} \text{ — Linguaggi}$$

⇒ **stringhe**:

$$\{e, ab, a^2b^2, \dots\} \text{ — Fondamenti}$$

programma — Linguaggi

⇒ **linguaggio**:

$$\{a^n b^n \mid n \in \mathbb{N}\} \text{ — Fondamenti}$$

$$\{\text{PL: insieme di programmi}\} \text{ — Linguaggi}$$

Nell'ambito dei linguaggi di programmazione gli stessi concetti prendono nomi differenti:

- **parola = lessema**: parola con significato specifico.
Nella grammatica corrisponde ad un terminale, ovvero l'unità minima sintattica.

Esempio: lessema

index = 2

⇒ index, =, 2 sono lessemi

- **frasi = token**: elementi delle categorie sintattiche del linguaggio di programmazione.
Nella grammatica corrispondono alle sequenze generate dai simboli non terminali.

Esempio: token

index = 2

⇒ si tratta di un assegnamento/comando

C'è bisogno di strumenti formali per descrivere parole e frasi legali di un linguaggio, che permettano in modo automatico di verificare l'appartenenza o meno del linguaggio.

Nei linguaggi di programmazione, il linguaggio dei lessemi è in generale sempre un linguaggio regolare, quindi riconosciuto da un automa a stati finiti.

Definizione: riconoscitore

Uno strumento di riconoscimento legge in input stringhe sull'alfabeto del linguaggio e decide se la stringa appartiene o meno al linguaggio.

Es: automa stati finiti → analisi lessicale per il riconoscimento di lessemi

Definizione: generatore

Uno strumento che genera stringhe di un linguaggio.

Si può determinare se la sintassi di una particolare frase è sintatticamente corretta confrontandola con la struttura del generatore (parser).

2.1.1 Grammatiche context-free

Le grammatiche CF permettono di descrivere la sintassi dei linguaggi di programmazione.

Definizione: grammatica CF

Una **grammatica context-free** è definita dalla quadrupla $G = \langle V, T, P, S \rangle$

- ▷ **V**: insieme finito di variabili (simboli non terminali) → **categorie sintattiche**
- ▷ **T**: insieme finito di simboli ($V \cap T = \emptyset$) → **vocabolario**
- ▷ **P**: insieme finito di produzioni. → **collezione di regole di formazione/composizione**
 Ogni produzione è definita nella forma $A \rightarrow \alpha$, dove:
 - ◊ $A \in V$: variabile
 - ◊ $\alpha \in (V \cup T)^*$
- ▷ **S**: simbolo iniziale → **categoria delle frasi**

Nella grammatica i **simboli terminali** costituiscono le parole/lessemi del linguaggio (vocabolario), mentre i **simboli non terminali** sono le categorie sintattiche (diversi tipi di elementi).

Le **produzioni** sono le regole di composizione degli elementi di una frase.

Il **linguaggio** è l'insieme di tutte le possibili frasi (stringhe di simboli non terminali).

Il fatto che il linguaggio sia context-free costituisce un vincolo sulle produzioni perché in un **qualunque contesto** troviamo quel non terminale, lo possiamo sostituire/rimpiazzare applicando una delle produzioni per il non terminale.

In altre parole, il contesto in cui si trova un non terminale A , non pone un vincolo sulle produzioni per A che noi possiamo applicare.

Nei linguaggi di programmazione questo significa che il modo con cui possiamo sviluppare, per sostituzione, una categoria sintattica è indipendente dal contesto.

Questo è un vantaggio perché dato un programma abbiamo il problema di capire quale grammatica può averlo generato (parsing), e per le grammatiche CF esistono strumenti automatici efficienti di parsing.

Dal punto di vista della complessità del parsing, questa dipende dal numero di simboli terminali della grammatica e dal grado di non determinismo delle produzioni.

Un linguaggio può essere generato da diverse grammatiche, quindi ai fini dell'efficienza del linguaggio è importante come si sceglie la grammatica.

2.1.2 Notazione BNF

La BNF è un metalinguaggio usato per descrivere linguaggi di programmazione.

Definizione: notazione BNF

Notazione è usata per descrivere grammatiche CF, dove si usano intere parole come simboli terminali.

I simboli non terminali sono identificati racchiudendoli tra parentesi angolate $\langle \rangle$ e per le produzioni si usa il simbolo $::=$ al posto della freccia.

$$\langle A \rangle ::= \alpha \langle B \rangle \gamma \mid \alpha$$

$$\langle B \rangle ::= \epsilon \mid \beta_1 \mid \beta_1 \mid \beta_2$$

2.1.3 Definizione non formale

Un linguaggio può essere descritto anche in modo informale, descrivendo i costrutti base:

- niente dichiarazioni
- solo variabili ed espressioni booleane
- assegnamento e composizione sequenziale
- comando condizionale
- comando iterativo (loop)

Il linguaggio è descritto su più livelli di astrazione, ognuno descritto da una grammatica.

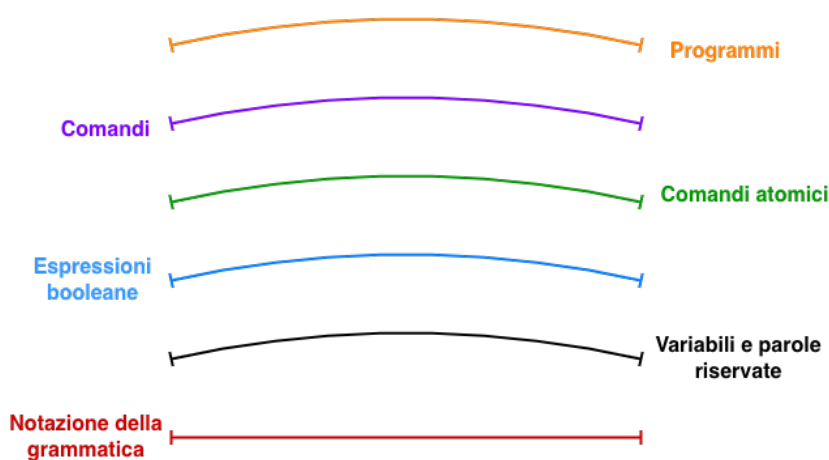


Figura 2.1: livelli di un PL

$\langle \text{program} \rangle$	$S \rightarrow C$
$\langle \text{com} \rangle$	$C \rightarrow A \mid C ; C \mid \text{if } B \text{ then } C \text{ else } C \mid \text{while } B \text{ do } C$
$\langle \text{atomic com} \rangle$	$A \rightarrow v := B$
b-expr	$B \rightarrow \text{true} \mid \text{false} \mid v \mid (\text{not } B) \mid (B \text{ and } B) \mid (B \text{ or } B)$

Nelle espressioni abbiamo una notazione completamente "parentesitizzata" per evitare ambiguità.

L'ambiguità consiste nell'esistenza di più parse-tree per la stessa espressione.

2.2 Analisi semantica

L'analisi semantica verifica (identifica) i **vincoli sintattici** (es. n parametri attuali = n parametri formali).

Questi sono vincoli sul codice, quindi non catturati dalla grammatica e si definiscono **contestuali**.

- sintassi $\rightarrow$ grammatica CF
- semantica $\rightarrow$ tutti i processi di verifica ed esclusione che non sono descritti dalla sintassi

2.2.1 Induzione strutturale

L'**induzione matematica** permette di dimostrare proprietà di insiemi numerabili, dove possiamo identificare ogni elemento con un elemento naturale su cui facciamo induzione.

L'**induzione strutturale**, invece, permette di dimostrare proprietà di oggetti (es. insiemi) definiti in modo induttivo/ricorsivo.

Ciò significa che su qualunque oggetto definito in modo induttivo possiamo fare induzione strutturale, ovvero dimostrare proprietà sulla struttura dell'oggetto, determinata dalla costruzione induttiva.

Definizione: induzione strutturale

Per definire una proprietà P su tutti gli elementi di un insieme definito in modo strutturale si può utilizzare l'induzione strutturale da una grammatica bisogna:

1. (**caso base**): dimostrare la proprietà su tutti gli elementi base, cioè quelli che sono esplicitamente inseriti nell'insieme di stringhe generate dalla grammatica
2. (**passo induttivo**): dimostrare la proprietà sugli elementi composti sotto l'ipotesi induttiva che la proprietà valga per gli elementi che la compongono

Esempio:

Consideriamo l'insieme $X \subseteq \mathbb{N}$:

1. (caso base): $0 \in X$
2. (passo induttivo): se $n \in X$, allora $n + 3 \in X$

La proprietà è $X = 3\mathbb{N}$. La proprietà dimostrabile in modo strutturale è: $X \subseteq 3\mathbb{N}$:

1. (caso base): 0 è la base di X e $0 \in 3\mathbb{N}$
2. (passo induttivo): dimostriamo $\forall n. n \in X \Rightarrow n \in 3\mathbb{N}$
Prendiamo $n \in X$, l'ipotesi induttiva è la seguente:

$$n \in X \rightarrow 3\mathbb{N}$$

bisogna dimostrare che $n + 3 \in 3\mathbb{N}$.

Sappiamo che:

$$\begin{aligned} n \in 3\mathbb{N} &\Rightarrow \exists i. n = 3i \\ &\Rightarrow n + 3 = 3i + 3 = 3(i + 1) \in 3\mathbb{N} \end{aligned}$$

2.2.2 Semantica

Definizione: semantica

La **semantica** attribuisce un significato ad ogni frase sintatticamente corretta.

A seconda di quali strumenti matematici si utilizzano per descrivere i significati si ottengono tipi di semantiche diverse:

- ▷ **semantica denotazionale:**

Espressioni

Le espressioni sono oggetti sintattici usati per ragionare sui loro significati (valori). Per associare significati/valori alle espressioni, le espressioni devono essere **valutate**. Il valore ottenuto mediante la valutazione è chiamato **valore esprimibile**.

Definizione: valore esprimibile

Il **valore esprimibile** è un valore che può essere espresso attraverso la sintassi del linguaggio di programmazione e, in particolare, attraverso la sintassi delle espressioni.

$$\text{Eval} = \text{bool} \cup \text{init}$$

3.1 Caratteristiche delle espressioni

Le caratteristiche più importanti delle espressioni sono:

- ▷ **notazione**: specifica in che modo gli operandi e gli operatori vengono rappresentati e per indicare su quali operandi un operatore opera
- ▷ **regole di precedenza** (tra operatori)
- ▷ **regole di associatività** (dagli operatori)
- ▷ **presenza di side-effects**: qualunque effetto che non consiste puramente nella rappresentazione di valori
- ▷ **overloading degli operatori**: operatori che hanno significato diverso a seconda del tipo degli operatori a cui sono applicati
- ▷ **espressioni con tipi misti**: espressioni formate da tipi di valori diversi

3.1.1 Notazione

Dichiarazioni

4.1 Identificatori

Definizione: identificare

Un **identificatore** è una sequenza di caratteri usata per rappresentare e denotare un altro oggetto.

Gli identificatori sono, quindi, dei nomi usati per rifare altri elementi del linguaggio (es. procedure) o della macchina sottostante (celle di memoria) durante la computazione. Quando identificano celle di memoria sono anche dette **variabili**.



```
int pippo
```

```
int pluto() {  
    pippo = 1;  
}
```

Figura 4.1: identificatore

N.B.

Identificatore e oggetto denotato **non** sono la stessa cosa:

- ▷ un identificatore può denotare più elementi
- ▷ un elemento può essere denotato da più identificatori diversi (aliasing)

4.1.1 Aspetti progettuali

Gli identificatori non possono essere stringhe di caratteri qualunque, ma devono seguire delle regole in modo che il parser del linguaggio li possa riconoscere come tali.

Ogni linguaggio ha le sue regole che rispondono a una serie di quesiti:

◇ identificari sono o non sono case-sensitive?

In C, gli identificatori lo sono: Pippo $\neq$ pippo

◇ **esistono parole riservate al linguaggio?**

Le parole riservate sono stringhe di caratteri che il linguaggio non permette di usare come identificatori perché hanno associati a specifici significati

◇ **ci sono vincoli di lunghezza?**

Anche se gli identificatori lunghi sono poco pratici, esistono linguaggi che impongono dei limiti: FORTRAN ha un massimo di 31 caratteri, mentre in C non ci sono limiti

◇ **ci sono caratteri speciali per specificare gli identificatori?**

In PHP gli identificatori devono iniziare con il simbolo \$

4.1.2 Bindings - legami

Il concetto **binding** deriva dalla matematica, in cui i nomi vengono usati per denotare oggetti complessi.

Prima, però, di poterli usare, dobbiamo creare il legame, ovvero "dichiarare" il nome e definire il suo significato.

Definizione: binding occurrences (creazione binding)

Viene definito il **nome** (identificatore) e il suo **significato** (denotazione).

$$\left(\sum_{i=1}^n \left(\sum_{j=1}^m a_{ij} \dots b_{jk}\right)\right)$$

In matematica, ogni volta che si pone un problema dicendo "sia x ...", si sta creando un **binding** tra il **significato dichiarato** e il **nome** che verrà utilizzato nella specifica del problema.

In logica, i binding sono creati dai **quantificatori**, mentre in programmazione dalle **dichiarazioni**.

Sucessivamente, il nome può essere usato per riferirsi/rappresentare il suo significato e sono definiti **occorrenze di applicazione**.

Definizione: applied occurrences (applicazione binding)

Viene usato il **nome** (identificatore) per accedere al suo **significato** (denotazione).

In matematica, i binding vengono creati prima dell'uso, anche se è possibile crearli dopo.
[2ex] Anche in programmazione non è sempre necessario creare i binding prima, ma per motivi di compilazione molti PL impongono che gli identificatori sono definiti prima di essere usati.

Un nome non dichiarato (non è stato descritto il significato) è un nome che può rappresentare qualunque cosa in quel punto.

Definizione: free occurrences

Viene usato un **nome** (identificatore) il cui **significato** non è stato definito.

Da un punto di vista del costrutto, le occorrenze libere e quelle applicate sono identiche, la differenza consiste nell'essere nel raggio di azione (scope) di una occorrenza di definizione.

Definizione: scope di un binding

Lo **scope** definisce lo spazio in cui può usare un nome per rappresentare il significato associato da un binding.

4.1.2.1 Bindings nei PL**4.1.2.2 Tipi di bindings****4.1.2.3 Tempi di bindings****4.2 Semantica delle dichiarazioni**